

day 07 | 250912 F

heretofore

We talked about plotting last time, but I wanted to make a slight distinction in the way that we generated that data. We used the array-wise math that numpy offers us, but I felt like I skipped over a point I wanted to make. The numpy method is much much faster than using a `for` loop to do this, and we employ this so often that I wanted to say something about it before I forgot.

herein

Now that we have the basics of plotting we need to practice this with some real data and see what trouble we run into. So today we will import and trim data, and then prepare some plots based on that data. We also want to check in and follow up on the last few assignments that you have, namely finishing printing out Pascal's triangle as well as finding a python graphing module and showing that off to the class.

To import some data to python, there are a variety of ways, each depending on how complex the data is. Today, we will start with a simple example of two columns of data in a file. The file extension is readable by python as simply a comma separated value or csv file. Many files that have various extensions are like this, and many/most experimental apparatus will have the ability to export data in such a format.

After loading the data using `np.loadtxt()`, we have to take the transform of this file using `data.T`, and then we are able to use `matplotlib` to plot the data just like before.

Next, we will choose a more complicated file, and talk through how we can import it using `pandas`. `Pandas` is a python library specifically for handling more complicated data files than numpy is good for. Notice for instance that `pandas` handles text headers to give a column a particular name. Each `pandas` column acts like an individual numpy array.

In this case we have a file that has a large header that we need to deal with, as well as a format problem. We will use a statement like this to load the data into what is called a "Data Frame", which is really just python's word for a database.

```
data = pd.read_csv('filename.txt', header=55, sep='\t',  
encoding='windows-1252')
```

Now some of this is specific to this particular file, but these are some of the options you may need to use from time to time. Each column in this data file already has a

name

This will lead us into a discussion of formats and what you can do with them. For example did you know that you can unzip a word document?

From here, work exercises 3.1 and 3.2.

hence

We did not get to all of this, so we are going to need another day to complete the lesson. My apologies, but the first part of this took too much time to spell out. But we will finish the lesson here next time.

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
In [3]: x_data = np.linspace(0,10)
```

```
In [5]: y_data = 5*x_data-3
```

```
In [6]: y_data
```

```
Out[6]: array([-3.          , -1.97959184, -0.95918367,  0.06122449,  1.08163265,
        2.10204082,  3.12244898,  4.14285714,  5.16326531,  6.18367347,
        7.20408163,  8.2244898 ,  9.24489796, 10.26530612, 11.28571429,
        12.30612245, 13.32653061, 14.34693878, 15.36734694, 16.3877551 ,
        17.40816327, 18.42857143, 19.44897959, 20.46938776, 21.48979592,
        22.51020408, 23.53061224, 24.55102041, 25.57142857, 26.59183673,
        27.6122449 , 28.63265306, 29.65306122, 30.67346939, 31.69387755,
        32.71428571, 33.73469388, 34.75510204, 35.7755102 , 36.79591837,
        37.81632653, 38.83673469, 39.85714286, 40.87755102, 41.89795918,
        42.91836735, 43.93877551, 44.95918367, 45.97959184, 47.          ])
```

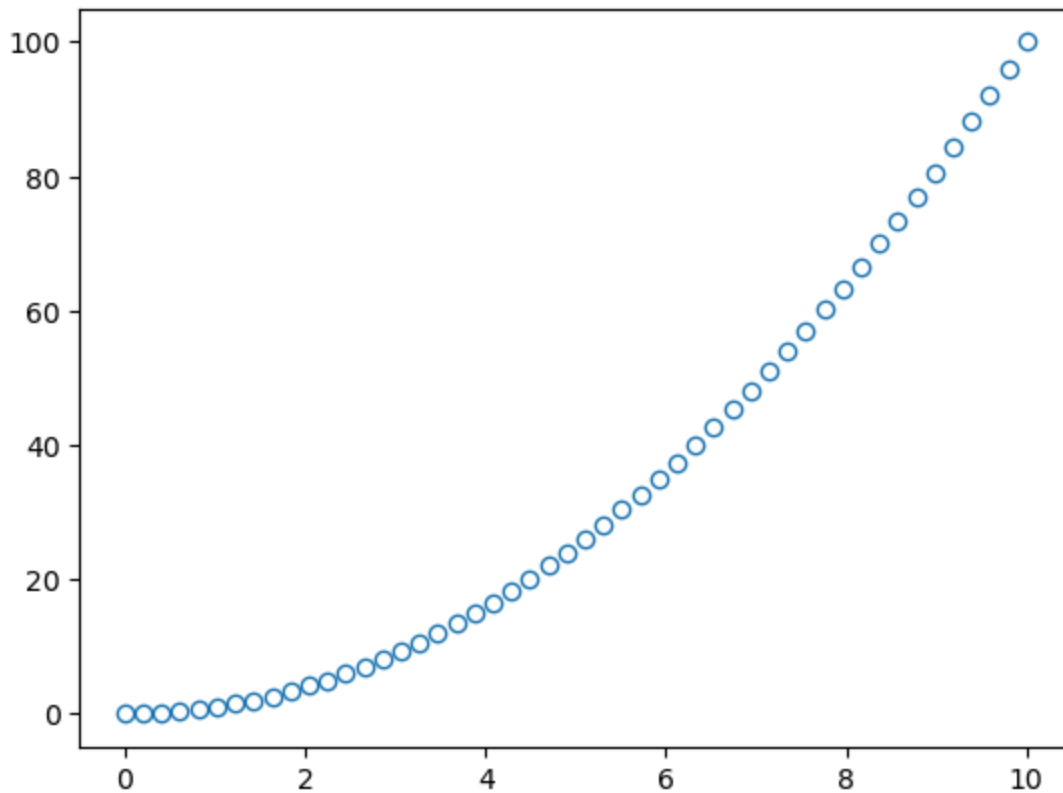
```
In [10]: x_data2 = []
y_data2 = []

for x in np.linspace(0,10):
    x_data2.append(x)
    y_data2.append(x**2)

fig0 = plt.figure()
ax0 = fig0.add_subplot()

ax0.plot(x_data2, y_data2, 'o', mfc='none')
```

```
Out[10]: [<matplotlib.lines.Line2D at 0x7be8142edca0>]
```



```
In [11]: data = np.loadtxt('../data/150407-02.dat', skiprows=1)
```

```
In [12]: data
```

```
Out[12]: array([[20.  , 28.  ],
                [20.01, 17.  ],
                [20.02, 23.  ],
                ...,
                [74.98,  8.  ],
                [74.99, 12.  ],
                [75.  ,  0.  ]])
```

```
In [13]: data = data.T
```

```
Out[13]: array([[20.  , 20.01, 20.02, ..., 74.98, 74.99, 75.  ],
                [28.  , 17.  , 23.  , ...,  8.  , 12.  ,  0.  ]])
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```